



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 3

**“Convertidor BCD a 7 Segmentos” y “Contador
Hexadecimal”**

Grupo: 6CM1

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 13 de febrero de 2025

INTRODUCCIÓN

Convertidor BCD a 7 Segmentos

Un convertidor BCD (Binary Coded Decimal) a 7 segmentos es un circuito combinacional que "traduce" un número en binario de 4 bits (que representa los dígitos del 0 al 9) a las señales necesarias para encender los segmentos de un display.

Funcionamiento.

- **Entrada:** Recibe 4 líneas de datos ($2^0, 2^1, 2^2, 2^3$).
- **Procesamiento:** Internamente tiene una matriz de compuertas lógicas que determina qué segmentos deben encenderse para formar el número.
- **Salida:** Tiene 7 salidas (etiquetadas de la a a la g), una para cada barra del display.

Tipos de conexión:

Es vital saber qué tipo de display se tiene para elegir el convertidor correcto:

- **Ánodo Común:** El display tiene el positivo conectado a un solo punto; el convertidor debe enviar "ceros" (GND) para encender los segmentos.
- **Cátodo Común:** El display tiene el negativo conectado a tierra; el convertidor debe enviar "unos" (VCC) para encenderlos.

Contador Hexadecimal

A diferencia de los contadores de década (que se resetean al llegar a 9), un contador hexadecimal aprovecha al máximo sus 4 bits de salida, contando desde el 0 hasta el 15 ($2^4 = 16$ estados).

Secuencia de conteo:

En electrónica digital, después del 9 no pasamos al 10 decimal, sino que seguimos con letras: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.

Características:

- **Cascada:** Tienen una salida llamada Carry (Acarreo) que envía un pulso cuando el contador pasa de 'F' a '0', lo que permite conectar otro contador para representar números más grandes (como 00 a FF).
- **Reset y Preset:** Permiten forzar el conteo a cero o cargar un número específico para empezar a contar desde ahí.
- **Chip de referencia:** El 74LS191 o el 74LS193 son ejemplos comunes de contadores hexadecimales síncronos Up/Down.

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 3, el circuito a armar es el siguiente:

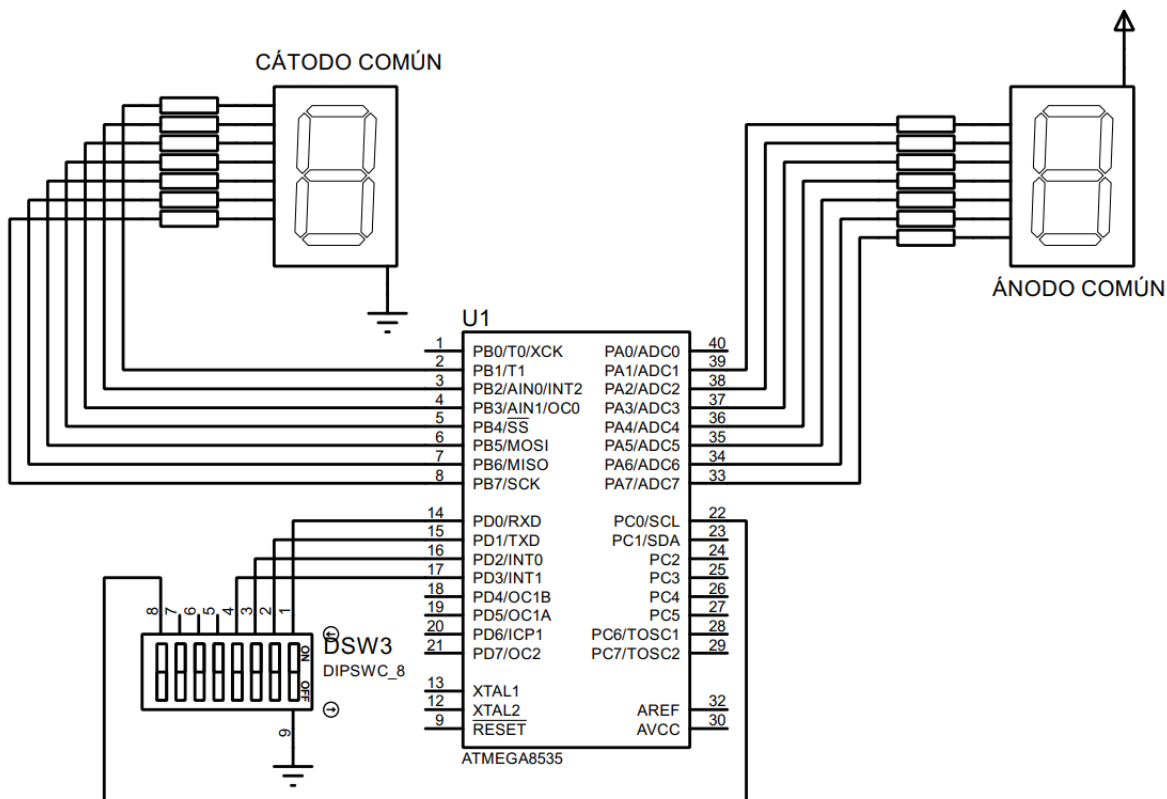


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen del circuito, el registro B se configura como de salida (PB1 – PB7), también el A (PA1 – PA7), mientras que el registro D se configura como de entrada (PDO – PD3) y por último el registro C también como de salida (PC0) y estos últimos con las resistencias de pull-up activadas.

Ports Settings

PORTA Port B Port C Port D

	Data Direction	Pullup/Output Value	
Bit 0	In	T	Bit 0
Bit 1	Out	0	Bit 1
Bit 2	Out	0	Bit 2
Bit 3	Out	0	Bit 3
Bit 4	Out	0	Bit 4
Bit 5	Out	0	Bit 5
Bit 6	Out	0	Bit 6
Bit 7	Out	0	Bit 7

Imagen 2. Configuración del registro A.

Ports Settings

PORTA Port B Port C Port D

	Data Direction	Pullup/Output Value	
Bit 0	In	T	Bit 0
Bit 1	Out	0	Bit 1
Bit 2	Out	0	Bit 2
Bit 3	Out	0	Bit 3
Bit 4	Out	0	Bit 4
Bit 5	Out	0	Bit 5
Bit 6	Out	0	Bit 6
Bit 7	Out	0	Bit 7

Imagen 3. Configuración del registro B.

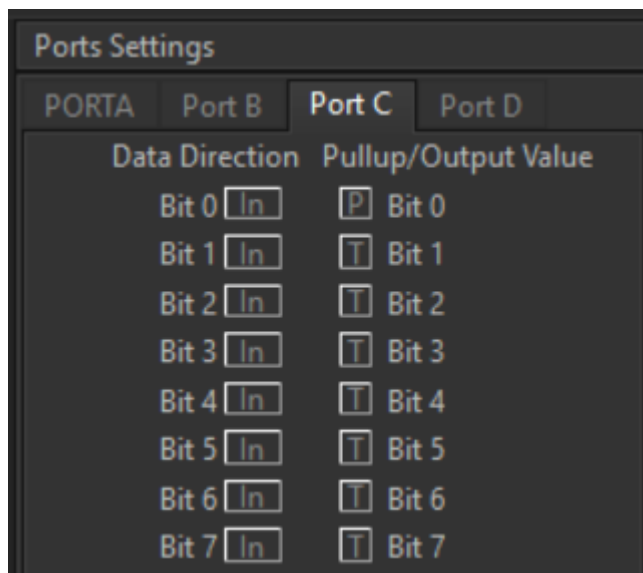


Imagen 4. Configuración del puerto C.

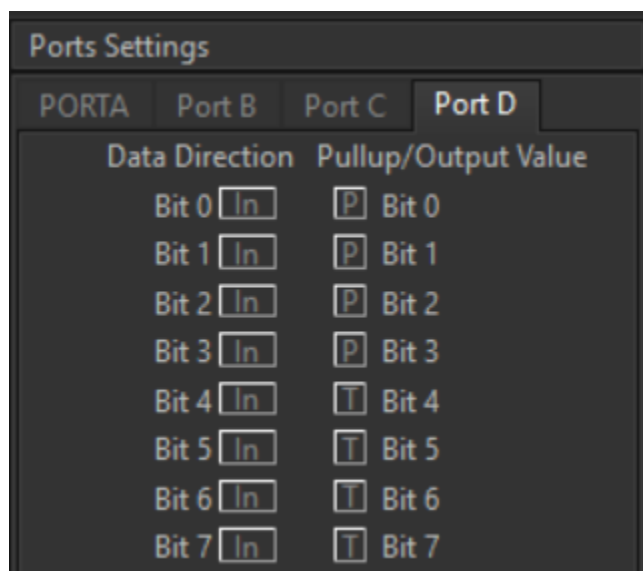


Imagen 5. Configuración del puerto D.

Para poder hacer el convertidor BCD a 7 segmentos y el contador hexadecimal, es necesario interpretar el número correspondiente del display como un número hexadecimal, para ello se realizó la siguiente tabla:

Número Display	Combinaciones								Valor hexadecimal
	.	g	f	e	d	c	b	a	
0	0	0	1	1	1	1	1	1	0x3f
1	0	0	0	0	0	1	1	0	0x06
2	0	1	0	1	1	0	1	1	0x5b
3	0	1	0	0	1	1	1	1	0x4f
4	0	1	1	0	0	1	1	0	0x66

5	0	1	1	0	1	1	0	1	0x6d
6	0	1	1	1	1	1	0	1	0x7d
7	0	0	0	0	0	1	1	1	0x07
8	0	1	1	1	1	1	1	1	0x7f
9	0	1	1	0	1	1	1	1	0x6f
A	0	1	1	1	0	1	1	1	0x77
B	0	1	1	1	1	1	0	0	0x7c
C	0	0	1	1	1	0	0	1	0x39
D	0	1	0	1	1	1	1	0	0x5e
E	0	1	1	1	1	0	0	1	0x79
F	0	1	1	1	0	0	0	1	0x71

Tabla 1. Representación hexadecimal para los números del 0 al 9 y del A a la F en hexadecimal.

Debido a que el pin PA0 del ATMEGA se rompió, se tuvieron que ajustar los valores hexadecimales, recorriendo los bits un bit hacia la izquierda, por tanto, estos nuevos valores se indican en la siguiente tabla:

Número Display	Combinaciones								Valor hexadecimal
	g	f	e	d	c	b	a		
0	0	1	1	1	1	1	1	0	0x7e
1	0	0	0	0	1	1	0	0	0x0c
2	1	0	1	1	0	1	1	0	0xb6
3	1	0	0	1	1	1	1	0	0x9e
4	1	1	0	0	1	1	0	0	0xcc
5	1	1	0	1	1	0	1	0	0xda
6	1	1	1	1	1	0	1	0	0xfa
7	0	0	0	0	1	1	1	0	0x0e
8	1	1	1	1	1	1	1	0	0xfe
9	1	1	0	1	1	1	1	0	0xde
A	1	1	1	0	1	1	1	0	0xee
B	1	1	1	1	1	0	0	0	0xf8
C	0	1	1	1	0	0	1	0	0x72
D	1	0	1	1	1	1	0	0	0xbc
E	1	1	1	1	0	0	1	0	0xf2
F	1	1	1	0	0	0	1	0	0xe2

Código del circuito de CodeVision

```

1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4      Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Pr  ctica 3 'Convertidor BCD a 7 Segmentos' y 'Contador Hexadecimal'
8  Version : 1.0
9  Date    : 10/02/2026
10 Author  : Garc  a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type      : ATmega8535
16 Program type   : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model   : Small
19 External RAM size : 0
20 Data Stack size : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25 #define MOD0 PINC.0

```

```

27 // Declare your global variables here
28
29 void main(void) {
30     // Declare your local variables here
31     unsigned char valor_entrada;
32     const char bcd[10] = {0x7e, 0x0c, 0xb6, 0x9e, 0xcc, 0xda, 0xfa, 0xe, 0xfe, 0xde};
33     const char hex[16] = {0x7e, 0x0c, 0xb6, 0x9e, 0xcc, 0xda, 0xfa, 0xe, 0xfe, 0xde, 0xee, 0xf8, 0x72, 0xbc, 0xf2, 0xe2};
34
35     // Input/Output Ports initialization
36     // Port A initialization
37     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
38     DDRA=(1<<DDA7) | (1<<DDA6) | (1<<DDA5) | (1<<DDA4) | (1<<DDA3) | (1<<DDA2) | (1<<DDA1) | (1<<DDA0);
39     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
40     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
41
42     // Port B initialization
43     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
44     DDRB=(1<<ddb7) | (1<<ddb6) | (1<<ddb5) | (1<<ddb4) | (1<<ddb3) | (1<<ddb2) | (1<<ddb1) | (1<<ddb0);
45     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
46     PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
47
48     // Port C initialization
49     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
50     DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
51     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=P
52     PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (1<<PORTC0);

```

```

54 // Port D initialization
55 // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
56 DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
57 // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=P Bit2=P Bit1=P Bit0=P
58 PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (1<<PORTD3) | (1<<PORTD2) | (1<<PORTD1) | (1<<PORTD0);
59
60 // Timer/Counter 0 initialization
61 // Clock source: System Clock
62 // Clock value: Timer 0 Stopped
63 // Mode: Normal top=0xFF
64 // OC0 output: Disconnected
65 TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
66 TCNT0=0x00;
67 OCR0=0x00;

```



```

69 // Timer/Counter 1 initialization
70 // Clock source: System Clock
71 // Clock value: Timer1 Stopped
72 // Mode: Normal top=0xFFFF
73 // OC1A output: Disconnected
74 // OC1B output: Disconnected
75 // Noise Canceler: Off
76 // Input Capture on Falling Edge
77 // Timer1 Overflow Interrupt: Off
78 // Input Capture Interrupt: Off
79 // Compare A Match Interrupt: Off
80 // Compare B Match Interrupt: Off
81 TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
82 TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
83 TCNT1H=0x00;
84 TCNT1L=0x00;
85 ICR1H=0x00;
86 ICR1L=0x00;
87 OCR1AH=0x00;
88 OCR1AL=0x00;
89 OCR1BH=0x00;
90 OCR1BL=0x00;

92 // Timer/Counter 2 initialization
93 // Clock source: System Clock
94 // Clock value: Timer2 Stopped
95 // Mode: Normal top=0xFF
96 // OC2 output: Disconnected
97 ASSR=0<<AS2;
98 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
99 TCNT2=0x00;
100 OCR2=0x00;

102 // Timer(s)/Counter(s) Interrupt(s) initialization
103 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);

105 // External Interrupt(s) initialization
106 // INT0: Off
107 // INT1: Off
108 // INT2: Off
109 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
110 MCUCSR=(0<<ISC2);

112 // USART initialization
113 // USART disabled
114 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCS2) | (0<<RXB8) | (0<<TXB8);

116 // Analog Comparator initialization
117 // Analog Comparator: Off
118 // The Analog Comparator's positive input is
119 // connected to the AIN0 pin
120 // The Analog Comparator's negative input is
121 // connected to the AIN1 pin
122 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
123 SFIOR=(0<<ACME);

125 // ADC initialization
126 // ADC disabled
127 ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);

129 // SPI initialization
130 // SPI disabled
131 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);

133 // TWI initialization
134 // TWI disabled
135 TWCR=(0<<TWIEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

```



```
137 while (1) {  
138     // Place your code here  
139     valor_entrada = PIND & 0x0f; // Enmascarar los 4 bits menos significativos  
140  
141     if (MOD0) {  
142         if (valor_entrada < 10) {  
143             PORTB = bcd[valor_entrada];  
144             PORTA = ~bcd[valor_entrada];  
145         } else {  
146             // Mostrar la 'e' de error  
147             PORTB = 0xf2;  
148             PORTA = ~0xf2;  
149         }  
150     } else {  
151         PORTB = hex[valor_entrada];  
152         PORTA = ~hex[valor_entrada];  
153     }  
154 }  
155 }
```

Circuito físico

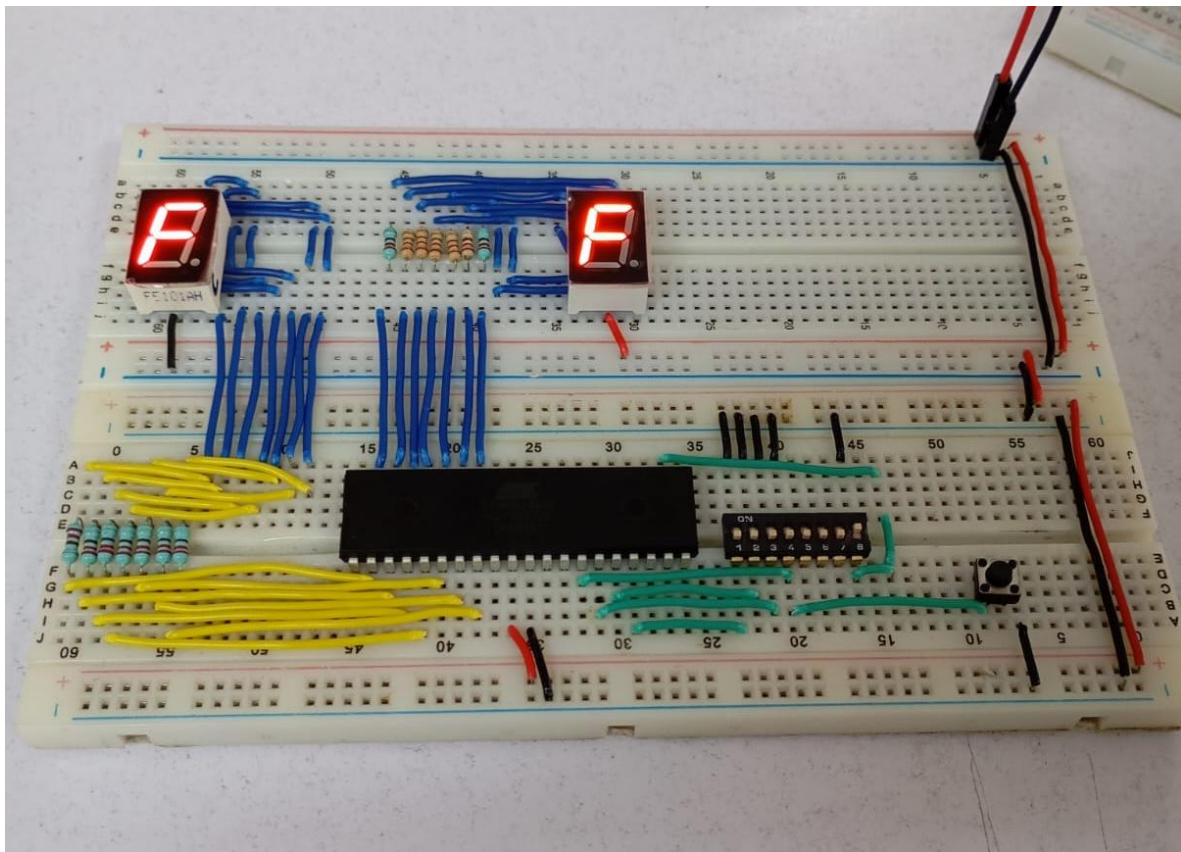


Imagen 3. Convertidor BCD a 7 segmentos.

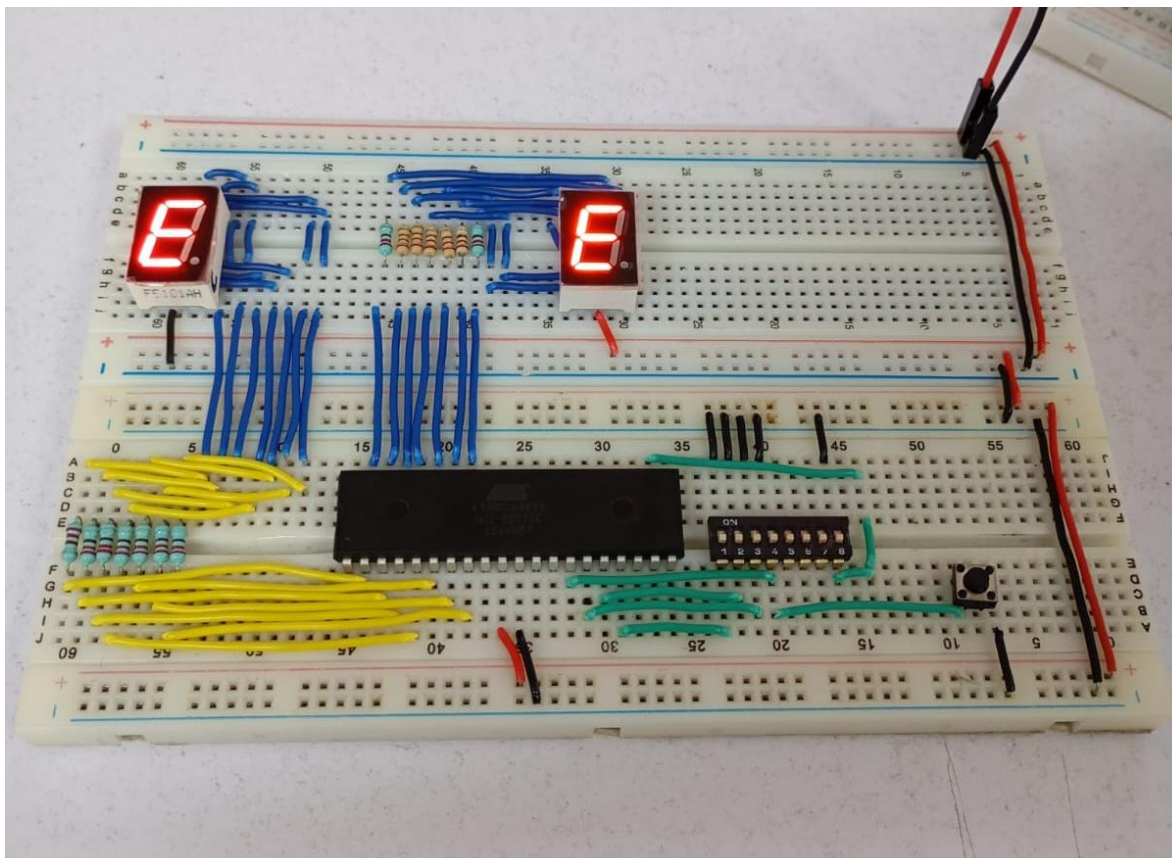


Imagen 4. Contador hexadecimal de 0 a F.

CONCLUSIÓN

- **García Escamilla Bryan Alexis**

En esta práctica se usa por primera vez el display de 7 segmentos, y su uso es muy interesante puesto que se usan números hexadecimales para facilitar el mostrar cualquier cosa en ellos. En este caso como ya se sabía que es lo que se tenía que mostrar, se usó un arreglo con todos los números hexadecimales para los números desde el cero hasta el nueve y posteriormente desde la A a la F en hexadecimal. El truco en esto es ir recorriendo el arreglo y asignarle al puerto de salida el número hexadecimal correspondiente dependiendo la combinación binaria introducida por medio de un dip-switch.

CodeVision facilita mucho la comparación de números decimales, binario y hexadecimales entre ellos, ya que todo se maneja en binario internamente, es por ello por lo que estando en el modo BCD se puede hacer la comparación `valor_entrada > 10` comparando la combinación del dip-switch con el número 10.

En esta práctica se usan ambos tipos de display, cátodo y ánodo común, y por convención se usa el primero pues funciona con unos lógicos, y en el caso del ánodo con ceros, es decir que los números hexadecimales en el arreglo se tienen que negar (invertir ceros y unos), para ello CodeVision nos permite usar la virgulilla (~) antes de obtener el número para invertirlo

(PORTX = ~bcd[valor_entrada]), permitiéndonos usar el mismo arreglo y ambos displays.

- **Meléndez Macedonio Rodrigo**

Para el desarrollo de esta práctica se nos pidió usar el display de 7 segmentos para mostrar valores del 0 al 9, y del mismo modo, para mostrar valores hexadecimales de la A a la F respectivamente. Para ello hicimos ambos contadores en un mismo circuito e implementamos un botón que se va a encargar de hacer el cambio de “BCD a 7 segmentos” al “Contador hexadecimal”. Es importante mencionar que también implementamos display de cátodo y ánodo común para esta práctica.

En cuanto al desarrollo del código este me sorprendió mucho pues CodeVisionAVR ofrece la ventaja de que puedes crear un arreglo con los valores de cada número del contador, ya sea BCD o hexadecimal, de este modo, lo que hicimos fue poner un condicional “if” con el valor de entrada menor a 10 para el caso del “BCD a 7 segmentos” ya que en caso de que se supere ese valor, soltara error en el display con la letra “E”. Y, por último, para el caso del “Contador hexadecimal” simplemente colocamos el cambio de los valores del puerto dentro de un “else” para cuando cambiemos de modo con ayuda del botón.

BIBLIOGRAFÍA

- (2025). *BCD to 7-Segment Decoder*. GeeksforGeeks. Recuperado el 09 de febrero de 2026. <https://www.geeksforgeeks.org/bcd-to-7-segment-decoder/>